



ECOLE NATIONAL SUPERIEURE D'INFORMATIQUE ET D'ANALYSE
DES SYSTEMES

FILIÈRE INGÉNIERIE DES SYSTÈMES EMBARQUÉS ET MOBILES

Rapport de realisation du compilateur ZX

Documentation du compilateur ZX

Réalisé par :

El Hari Chadi
Moumou Chaimae
Rahmouni Meryem
Dahbi Abdelmalek

Encadré par :

M. TABII Youness

Année universitaire : 2020-2021

Résumé

L'apparition des langages de haut niveau que l'assembleur pendant les années 50 est directement liée au besoin d'abstraction de certaines particularités propres à la machine et de rapprocher les langages de programmation au langage humain. Ce besoin d'abstraction a automatiquement donné naissance au concept de compilateur. Un compilateur est donc un outil qui traduit un langage de haut niveau vers un langage machine ou intermédiaire en l'optimisant dans un sens de continuité de l'automatisation des procédés. L'architecture la plus générale d'un compilateur est la suivante :

- I Analyse lexicale
- II Analyse syntaxique
- III Analyse sémantique
- IV Génération de code

La réalisation de ce projet comprends la concrétisation des deux modules d'analyse lexicale et syntaxique pour notre compilateur.

Table des matières

1	Introduction	1
2	Présentation du compilateur ZX	2
2.0.1	Langage source et cible	2
2.0.2	Élaboration du compilateur	4
2.0.3	Différentes parties d'un compilateur	4
2.0.4	Analyse lexicale	5
2.0.5	Analyse syntaxique	6
3	Analyseur lexical	7
4	Analyseur syntaxique	8
5	Réalisation du compilateur	10
5.0.1	Extraits du code	10
5.0.2	Exécution	14
6	Conclusion	17

Table des figures

2.1	Chaine de compilation	4
5.1	Déclarations dans le fichier header	10
5.2	Fonction fetchT().	11
5.3	Fonction skipWhiteSpace().	12
5.4	Procédure principale.	13
5.5	Fonction nextToken().	13
5.6	Code en langage ZX.	14
5.7	Sortie de l'analyse lexicale et syntaxique d'un code correct	15
5.8	Sortie de l'analyse lexicale et syntaxique d'un code incorrect	16

1 Introduction

La compilation est une transformation préservant l'équivalence d'un code écrit dans un Langage facilement compréhensible par l'humain (Langage de haut niveau) vers un langage machine ou intermédiaire (Langage de plus bas niveau). etc.

En général, un compilateur ne se contente pas de traduire un langage dans un autre, il est capable de signaler des erreurs de syntaxe, de sémantique (par exemple via une vérification de type) si possible de façon compréhensible par l'utilisateur, il fait des optimisations qui peuvent viser plusieurs objectifs parfois contradictoires : vitesse d'exécution, taille du code, utilisation de la mémoire (notamment pour les applications embarquées),

Concernant les erreurs, il existe traditionnellement quatre types à signaler :

- I Les conseils : ils donnent une information au programmeur pour probablement améliorer l'efficacité de son programme, mais n'arrêtent pas le processus de compilation ;
- II Les avertissements : ils avertissent le programmeur d'une erreur probable lors de l'exécution, mais n'arrêtent pas le processus de compilation ;
- III Les erreurs : elles indiquent au programmeur une faute dans le code source, faute qui doit être corrigée avant de pouvoir produire un code objet correct, mais la compilation continue ;
- IV Les erreurs fatales : elles stoppent immédiatement la compilation et empêchent le compilateur de produire le code objet.

Le présent rapport comporte la description des étapes entamées pour la réalisation des modules d'analyse lexicale et syntaxique concernant notre compilateur.

Les modules réalisés permettent de faire l'analyse lexicale et syntaxique d'un fichier source écrit en langage ZX que nous allons définir par la suite.

2 Présentation du compilateur ZX

2.0.1 Langage source et cible

Parler d'un compilateur est toujours associé à un langage source et un langage cible.

Langage source

Le langage source est le langage dans lequel est écrit le code source à analyser. Dans notre cas le fichier source doit être écrit dans le langage ZX afin de l'analyser. Le langage ZX est un langage proche de C et prend en considération tous les éléments d'un langage de programmation structuré.

Éléments du langage source

Le langage ZX prend en considération les éléments suivants :

- Une grammaire LL(1)
- Les opérateurs ('+', '-', '*', '/', '=')
- Les types (int, float, bool, void)
- Les structures conditionnelles (if ,then, else, else if)
- Les boucles (for ,while)
- Les instructions de saisie et d'affichage (printf, read)
- Les symboles ('', '||', '!=', '==', ';;', '<=', '>=', '(', ')', '{', '}', '%', "begin" , "end")

Grammaire

Les grammaires sont un élément important dans la définition d'un langage de programmation. Non seulement elles permettent de définir de façon formelle la syntaxe valide d'un code source, mais elles permettent aussi de faire générer automatiquement des analyseurs syntaxiques pour celles-ci. D'autre part, l'analyse syntaxique étant étroitement liée aux grammaires, l'utilisation en est largement faite. Il est donc important d'en comprendre la signification et la notation.

Grammaire du langage ZX

PROGRAM	::= {statementlist}{functiondefinition} EOF
FUNCTIONDEFINITION	::= TYPE ID (TYPE ID {,TYPE ID}) ({ begin) STATMENTLIST (} end)
DECL	::= TYPE ID ({,ID})
FUNCTIONCALL	::= ID (ASSIGNMENT{,ASSIGNMENT})
STATMEMNTLIST	::= {BLOCK}
BLOCK	::= ({ begin) STATMENTLIST (} end){ statement
STATMENT	::= IFSTATEMENT ELSESTATMENT RETURNSTATMENT; DECL; PRINTF; READ; FUNCTIONDEFINITION STATASSIGNMENT; FUNCTIONCALL; FORLOOPSTATEMENT WHILELOOPSTATEMENT
IFSTATEMENT	::= if(ASSIGNMENT) BLOCK
ELSESTATMENT	::= else if BLOCK
RETURNSTATMENT	::= return (ASSIGNMENT)
PRINTF	::= printf (CONST_STRING,ASSIGNMENT)
READ	::= read(CONST_STRING{,ID})
FORLOOPSTATEMENT	::= for (ASSIGNMENT,ASSIGNMENT,ASSIGNMENT) BLOCK
WHILELOOPSTATEMENT	::= while BLOCK
TYPE	::= bool int float void
STATASSIGNMENT	::= ID = ASSIGNMENT
ASSIGNMENT	::= ID = ASSIGNMENT EXPR
EXPR	::= SIMPEXPR ((== != <= >= < >) SIMPEXPR)
SIMPEXPR	::= {-} TERM {(+ -) TERM}
TERM	::= FACTOR {(* / &&) FACTOR }
FACTOR	::= CONST_INT CONST_FLOAT CONST_BOOLEAN FUNCTIONCALL ID (ASSIGNMENT)
ID	::= Lettre {Lettre Chiffre}
NUM	::= Chiffre {Chiffre}
Chiffre	::= 0 .. 9
Lettre	::= a b .. z A .. Z

Langage cible

Le langage cible est à déterminer pendant l'élaboration du générateur de code. Le langage cible pourrait être un langage intermédiaire qui serait interprété ou compilé par un autre éventuel compilateur, où un langage machine où assembleur de bas niveau utilisé par des machines cibles.

2.0.2 Élaboration du compilateur

Un compilateur effectue les opérations suivantes : analyse lexicale, décomposition analytique (parsing), analyse sémantique, génération de code et optimisation de code.

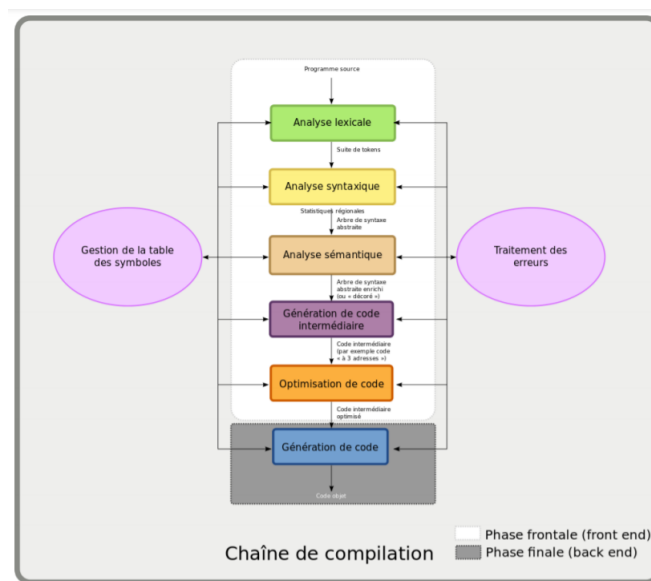


FIGURE 2.1 – Chaîne de compilation

2.0.3 Différentes parties d'un compilateur

Dans un compilateur, la partie avant (Front-end) est indépendante du langage cible, tandis que la partie arrière (Back-end) est indépendante du langage source.

- Partie avant (Front-end) : analyse
 - Lire le programme source et comprendre sa structure et sa signification
- Partie arrière (Back-end) : synthèse
 - Génère le programme équivalent en langage cible

De plus, un compilateur est composé de quatre modules en général :

- Module d'analyse lexicale
- Module d'analyse syntaxique
- Module d'analyse sémantique
- Module de génération de code et d'optimisation

2.0.4 Analyse lexicale

Cette phase a deux objectifs :

- Vérifier que le texte fourni en entrée peut se découper en mots, de sorte que chaque mot appartienne au langage utilisé.
- Découper le texte en unités lexicales, appelées également lexèmes ou Tokens, de sorte que chaque Token représente un mot du langage.

Tokens

Un Token est une série de caractères qui représente une unité d'information dans le code source. On distingue différents type de Tokens pour le langage XZ :

mots clés	Nom du Token
begin	KW_BEGIN
end	KW_END
void	KW_VOID
bool	KW_BOOLEAN
int	KW_INT
float	KW_FLOAT
true	CONST_BOOLEAN
false	CONST_BOOLEAN
printf	KW_PRINTF
read	KW_READ
if	KW_IF
else	KW_ELSE
for	KW_FOR
do	KW_DO
while	KW_WHILE
return	KW_RETURN

TABLE 2.1 – mot clés utilisés pour le langage

opération	Nom du Token
+	PLUS_TOKEN
-	SUB_TOKEN
*	MULT_TOKEN
/	DIV_TOKEN

TABLE 2.2 – opérations utilisés pour le langage

caractères spéciaux	Nom du Token
(	PO_TOKEN
)	PF_TOKEN
{	AO_TOKEN
}	AF_TOKEN
,	VIR_TOKEN
;	PV_TOKEN
=	EG_TOKEN
==	EQ
!=	NEQ
	OR
&&	AND
<	LSS
>	GRT
<=	LEQ
>=	GEQ

TABLE 2.3 – caractères spéciaux utilisés pour le langage

2.0.5 Analyse syntaxique

Cette phase suit immédiatement l'analyse lexicale. Ici, on analyse, non pas le texte fourni en entrée, mais le flux de Tokens généré par l'analyseur lexical. Cette phase a deux objectifs :

- Vérifier que le flux de Tokens forme une succession de phrases conforme à la syntaxe du langage utilisé.
- Transformer le flux de Tokens en un arbre appelé arbre de syntaxe abstraite (ASA)

Le problème de l'analyse syntaxique est celui de prendre une grammaire ,et de construire un algorithme capable, pour toute chaîne de terminaux, de décider si elle appartient ou non à $L(G)$ (notre langage). De plus, on demande de reconstruire un arbre de dérivation pour G en cas de réponse affirmative.

L'analyse descendante cherche à construire une dérivation gauche directement à partir du symbole initial, ou de façon équivalente, cherche à construire un arbre de dérivation à partir de la racine avec une exploration en profondeur d'abord de gauche à droite.

3 Analyseur lexical

L'analyseur lexical est le premier module d'un compilateur. Ce dernier reçoit le code source, le transforme en une série de Tokens et supprime les espaces et les commentaires. Si l'analyseur lexical détecte un Token qui n'est pas valide, un message d'erreur est généré.

Rôle de l'analyseur lexical

L'analyseur lexical exécute les tâches suivantes :

- Aide à identifier les Tokens dans une table de symboles.
- Supprime les espaces, les tabulations, les retours à la ligne et les commentaires du code source.
- Lie les messages d'erreurs avec le code source.
- Lit les caractères d'entrées du code source.

Erreurs lexicales

- Une séquence de caractères qui ne peut être scannée dans aucun token valide est une erreur lexicale ;
- Les erreurs lexicales sont rares, mais elles doivent quand même être traitées ;

Fonctions utilisées en analyse lexicale

Fonction	Description
Int nextToken()	Elle cherche le token suivant et retourne le code du token courant.
Void fetchT()	Cherche le token courant.
Void skipWhiteSpace()	Ignore les espaces, les tabulations, les retours à la ligne et les commentaires.
Int isNbr()	Vérifie si le token est un nombre.
Int isKeyword()	Vérifie si le token est un keyword, sinon, il est considéré identificateur.
Int isSymbole()	Vérifie les symboles reconnus par le langage.

TABLE 3.1 – Fonctions utilisées en analyse lexicale

4 Analyseur syntaxique

L'analyseur syntaxique structure la suite d'unités lexicales en catégories "grammaticales". On présente généralement cette structure à l'aide d'un arbre : l'arbre syntaxique. Rappelons que le partage du travail entre l'analyseur lexical et syntaxique est un peu arbitraire.

Construire l'analyseur syntaxique récursif LL(1) d'une grammaire consiste à effectuer successivement les étapes suivantes :

- Vérifier que la grammaire n'est pas ambiguë
- Réduire la grammaire si nécessaire
- Dérecursiver la grammaire si nécessaire
- Factoriser la grammaire
- Déterminer les non-terminaux produisant le vide
- Calculer les premiers et les suivants des non-terminaux
- En déduire les symboles directeurs des règles
- Écrire la procédure d'analyse associée à chaque non-terminal

Fonctions utilisées

Fonction	Symbole non-terminal
void program()	PROGRAM
void functionDefinition(int token, int state)	FUNCTIONDEFINITION
int functionCall()	FUNCTIONCALL
int statementList()	STATEMENTLIST
int block()	BLOCK
int statement(int token)	STATEMENT
void ifStatement()	IFSTATEMENT
void elseStatement()	ELSESTATEMENT
void forStmt()	FORLOOPSTATEMENT
void whileStmt()	WHILELOOPSTATEMENT
int returnStatement()	RETURNSTATEMENT
int printfStatement()	PRINTF
int readStatement()	READ
int type(int token)	TYPE
int statAssignment()	STATASSIGNMENT
int assignment(int token)	ASSIGNMENT
int expr()	EXPR
int simpExpr()	SIMPEXPR
int term()	TERM
int factor()	FACTOR
int decl()	DECL

TABLE 4.1 – Fonctions utilisées en analyse syntaxique

Chaque procédure analyse le Token reçu, fait appel au Tokens suivants nécessaires, et peut retourner une erreur grâce à la fonction "void error(int token, char* expected)" dans le cas échéant.

Chaque symbole non-terminal de la grammaire possède dans le tableau précédent une procédure d'analyse.

5 Réalisation du compilateur

Le code du compilateur est divisé en 2 fichiers :

- LexPars.c : Comprends le code des deux analyseurs lexical et syntaxique, ainsi que la procédure principale.
- LexPars.h : Fichier header, contenant les variables utilisées par notre compilateur.

5.0.1 Extraits du code

Les captures suivantes concernent des procédures et déclarations importantes du code du compilateur.

```
#ifndef PARSE_H
#define PARSE_H
#define lenValue 20
#define lenNom 20

#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

typedef struct {
    int code;
    char nom[lenNom];
    double floatValue;
    char value[lenValue];
    int intValue;
} Token;

#define AND 257
#define OR 258
#define EQ 259
#define NEQ 260
#define LEQ 261
#define GEQ 262
#define LSS 263
#define GRT 264
#define KW_BEGIN 265
#define KW_BOOLEAN 266
#define KW_DO 267
#define KW_ELSE 268
#define KW_END 269
#define KW_FLOAT 270
#define KW_FOR 271
#define KW_IF 272
#define KW_INT 273
#define KW_PRINTF 274
#define KW_RETURN 275

#define KW_VOID 276
#define KW_WHILE 277
#define CONST_INT 278
#define CONST_FLOAT 279
#define CONST_BOOLEAN 280
#define ID 281
#define KW_READ 282
#define CONST_STRING 283
#define SEP 284

Token Token_courant;

FILE* file;
const char *fileName = "test.zx";
FILE *fileTokens;
const char *tokens = "tokens.txt";
int ligne;

int col = 1;
char car_courant;
char str[50];
int i = 0;

void fetchT();
void LexError(char *message);
void skipWhiteSpace();
int isNbr();
int isKeyword();
int isSymbole();
int nextToken();

#endif
```

FIGURE 5.1 – Déclarations dans le fichier header


```

void fetchT(){
    int r;
    Token_courant.code=SEP;
    memset(Token_courant.nom, '\0', lenNom);
    memset(Token_courant.value, '\0', lenValue);
    Token_courant.floatValue=0;
    Token_courant.intValue=0;
    skipWhiteSpace();

    if (car_courant == EOF)
    {
        strcpy(Token_courant.nom, "EOF");
        Token_courant.code = EOF;
        printf("str:%s | code :%d\n",Token_courant.nom,Token_courant.code);
    }
    else if (isNbr()!=0){
        r=Token_courant.code;
        printf("data :[int]%d [float]%f | code:%d\n",Token_courant.intValue,Token_courant.floatValue,Token_courant.code);
    }
    else if (isKeyword()!=0){
        r=Token_courant.code;
        printf("str:%s | code :%d\n",Token_courant.nom,Token_courant.code);
    }
    else if (isSymbole()!=0){
        r=Token_courant.code;
        if (r==CONST_STRING){
            printf("CONST STRING:%d | %s\n",Token_courant.code,Token_courant.nom);
        }
        else{
            printf("Symbole:%d | %c\n",Token_courant.code,Token_courant.code);
        }
    }
    else{
        Token_courant.code=SEP;
        printf("SEP:%d\n",Token_courant.code);
    }
}
}

```

FIGURE 5.2 – Fonction fetchT().

```
void skipWhiteSpace()
{
    while (car_courant == '\n')
    {
        car_courant = fgetc(file);
        ligne++;
        col = 1;
    }

    while (car_courant == ' ' || car_courant == '\t')
    {
        car_courant = fgetc(file);
        if (car_courant == '\n'){
            while (car_courant == '\n')
            {
                car_courant = fgetc(file);
                ligne++;
                col = 1;
            }
        }
    }

    //Depasser les commentaires
    if (car_courant == '%')
    {
        while (fgetc(file) != '\n')
        {
        }
        col = 1;
        ligne++;
        car_courant = fgetc(file);
    }
}
```

FIGURE 5.3 – Fonction skipWhiteSpace().

```

int main(int argc, char** argv)
{
    fileTokens = fopen(tokens, _type: "w");
    file = fopen(fileName, _type: "r");
    if (file == 0)
    {
        fprintf(stderr,
            "Error : Can't open input file.\n",
            argv[1]);
        exit(status: -1);
    }

    car_courant = fgetc(file);
    program();
    fclose(fileTokens);
    return 0;
}

```

FIGURE 5.4 – Procédure principale.

```

int nextToken()
{
    do {
        fetchT();
    } while(Token_courant.code == SEP);

    return Token_courant.code;
}

```

FIGURE 5.5 – Fonction nextToken().

```
{
int d, blub1, mybool;
printf("program has started");
}
%this is a comment
int blub(int arg, float arg2) {
    int blub4, blub2, blub3;
    float blub2;
    blub1 = 23;
    blub2 = 17;
    mybool = true;
    for (i = 0, i < 20, i = i + 1){
        read("%d", d);
    }
    read();
    int ho() {
        blub4 = blub4 + 1;
    }
    blub(a+c, b,c-1);
    blub3 = 42;
    blub4=blub1*(blub2+blub3);
    printf("hello %s",c+d);
    while(c <= 40)
    {
        printf(d);
        c=c+1;
    }
    if (blub1 == blub2) {
        return arg2;
    }
    else {
        read("%s",d);
    }
}
}
```

FIGURE 5.6 – Code en langage ZX.

5.0.2 Exécution

Prenons le code ci-dessus comme entrée pour exécuter une analyse lexicale et syntaxique.

La sortie du programme de compilation est la suivante :

```
Symb:123 | {
  str:int | code :273
  str:d | code :281
  Symb:44 | ,
  str:blub1 | code :281
  Symb:44 | ,
  str:mybool | code :281
  Symb:59 | ;
  str:printf | code :274
  Symb:40 | (
    CONST STRING:283 | program has started
  Symb:41 | )
  Symb:59 | ;
  Symb:125 | }
  str:int | code :273
  str:blub | code :281
  Symb:40 | (
    str:int | code :273
    str:arg | code :281
    Symb:44 | ,
    str:float | code :270
    str:arg2 | code :281
    Symb:41 | )
  Symb:123 | {
    str:int | code :273
    str:blub4 | code :281
    Symb:44 | ,
    str:blub2 | code :281
    Symb:44 | ,
    str:blub3 | code :281
    Symb:59 | ;
    str:float | code :270
    str:blub2 | code :281
    Symb:59 | ;
    str:blub1 | code :281
    Symb:61 | =
    data :[int]23 [float]0.000000 | code:278
    Symb:59 | ;
    str:blub2 | code :281
    Symb:61 | =
    str:blub4 | code :281
    Symb:61 | =
    str:blub4 | code :281
    Symb:43 | +
    data :[int]1 [float]0.000000 | code:278
    Symb:59 | ;
    Symb:125 | }
    str:blub | code :281
    Symb:40 | (
      str:a | code :281
      Symb:43 | +
      str:c | code :281
      Symb:44 | ,
      str:b | code :281
      Symb:44 | ,
      str:c | code :281
      Symb:45 | -
      data :[int]1 [float]0.000000 | code:278
      Symb:41 | )
      Symb:59 | ;
      str:blub3 | code :281
      Symb:61 | =
      data :[int]42 [float]0.000000 | code:278
      Symb:59 | ;
      str:blub4 | code :281
      Symb:61 | =
      str:blub1 | code :281
      Symb:42 | +
      Symb:40 | (
        str:blub2 | code :281
        Symb:43 | +
        str:blub3 | code :281
        Symb:41 | )
        Symb:59 | ;
        str:printf | code :274
        Symb:40 | (
          CONST STRING:283 | hello %s
          str:c | code :281
          Symb:43 | +
          str:b | code :281
          Symb:44 | ,
          str:while | code :277
          Symb:40 | (
            str:c | code :281
            Symb:261 | =
            data :[int]40 [float]0.000000 | code:278
            Symb:41 | )
            Symb:123 | {
              str:printf | code :274
              Symb:40 | (
                str:d | code :281
                Symb:41 | )
                Symb:59 | ;
                str:c | code :281
                Symb:61 | =
                str:c | code :281
                Symb:43 | +
                data :[int]1 [float]0.000000 | code:278
                Symb:59 | ;
                Symb:125 | }
                str:if | code :272
                Symb:40 | (
                  str:blub1 | code :281
                  Symb:259 | =
                  str:blub2 | code :281
                  Symb:41 | )
                  Symb:123 | {
                    str:return | code :275
                    str:arg2 | code :281
                    Symb:59 | ;
                    Symb:125 | }
                    str:else | code :268
                    Symb:123 | {
                      str:read | code :282
                      Symb:40 | (
                        CONST STRING:283 | %s
                        Symb:44 | ,
                        str:d | code :281
                        Symb:41 | )
                        Symb:59 | ;
                        Symb:125 | }
                        str:EOF | code :-1
                        Process finished with exit code 0

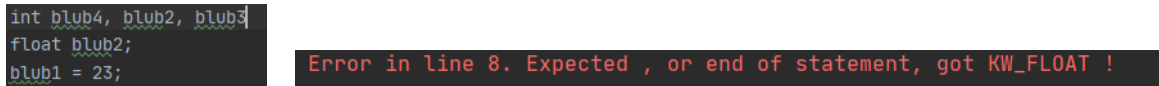
```

FIGURE 5.7 – Sortie de l’analyse lexicale et syntaxique d’un code correct

Erreurs

En cas d'erreur un message est renvoyé vers l'utilisateur indiquant la ligne d'erreur, le Token en question et le Token attendu.

Par exemple dans le cas suivant, un symbole ';' est manquant à la fin de l'expression. Un message d'erreur correspondant est affiché



```
int blub4, blub2, blub3;  
float blub2;  
blub1 = 23;
```

Error in line 8. Expected , or end of statement, got KW_FLOAT !

FIGURE 5.8 – Sortie de l'analyse lexicale et syntaxique d'un code incorrect

6 Conclusion

Nous avons réussi a la fin de ce projet à créer un analyseur lexical et syntaxique du langage ZX, dans une première étape vers la réalisation d'un compilateur complet pour ce langage. Ce projet nous a permis d'approfondir et concrétiser nos connaissances en termes de fonctionnements des langages de programmation structurels et des compilateurs. Le compilateur développé, qui colle aux spécifications imposées par le cahier des charges, peut et va être certainement amélioré dans les étapes à venir.